

Applied Data Science with R Capstone project

Tolulope Eribake

2025-04-27

Outline



- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary



- Analyze Bike-Share in 5 Cities
- Collect and process data
 - Seoul Bike Sharing Demand Dataset
 - OpenWeather's API 5-day Forecast
 - Global Bike-Share Dataset from Wikipedia
 - World Cities Data
- Perform EDA with SQL and ggplot on data
- Build Model to Predict Bike-Share
- Build Live Dashboard of Interactive Map and Visualization showing 5-day Weather Forecast and Estimated Bike-Share

Introduction



- To optimize availability and accessibility of rental bikes
- By meeting demand with adequate supply
- While minimizing cost of service
- Predict and show hourly bikes demand based on weather and time conditions
 - linear regressions using Tidymodels
 - interactive dashboard using R Shiny

Methodology



- Perform data collection
- Perform data wrangling
- Perform exploratory data analysis (EDA) using SQL and visualization
- Perform predictive analysis using regression models
 - Build the baseline model
 - Improve the baseline model
- Build Shiny dashboard app

Methodology

How we did it.

Data collection

- Web-scraped Global Bike-Sharing Systems data from Wikipedia leveraging rvest library's functions `read_html -> html_node -> html_table`
- Called OpenWeather's API to get 5-day Weather forecasts obtained riding on httr library's GET
- Used `download.file` to retrieve World Cities and Seoul Bike Sharing datasets

Data wrangling

- RENTED_BIKE_COUNT column has about 3% missing values (295 / 8760) which were dropped
- TEMPERATURE column had 11 missing values, all in summer, replaced with summer average temperature
- All character columns (categorical variables) one-hot-encoded
- All numerical variables/columns normalized
- All column names standardized
- Process the web-scraped bike sharing system dataset
- Remove undesired reference links using regular expressions
- Extract the numeric value using regular expressions

EDA with SQL

EDA done with SQL include:

- All-time high or busiest bike rental
- Hourly popularity and temperature by season
- Rental Seasonality
- Weather Seasonality
- Total Bike Count and City Info for Seoul
- Cities with comparable bike scale to Seoul's

EDA with data visualization

Charts plotted include:

- Scatter Plot of RENTED_BIKE_COUNT vs DATE
- Scatter Plot of RENTED_BIKE_COUNT vs DATE by HOUR
- Histogram with Kernel Density Curve of RENTED_BIKE_COUNT
- Scatter Plot of RENTED_BIKE_COUNT vs TEMPERATURE by SEASONS
- Boxplots of RENTED_BIKE_COUNT vs HOUR by SEASONS

Predictive analysis

- Several linear regression models were tested starting with simple linear model, polynomial model, model considering interaction of dependents, then improved upon this using 5-fold cross-validation and fine-tuned the penalty and mixture of L1 and L2 regularizations to obtain the best performing model
- RMSE and R-Squared were used to evaluate model minimizing test data RMSE well below 330 and maximizing R-Squared to over 0.77.

Build a R Shiny dashboard

Built leaflet based interactive map showing:

- Seoul, New York, Paris, Suzhou, and London
- Predicted bike-share demand
- Temperature trend line
- Humidity vs Bike demand with poly plot over next 5 days

Results



- Exploratory data analysis results
- Predictive analysis results
- A dashboard demo in screenshots

EDA with SQL

See appendix for queries.

Busiest bike rental times

A data.frame: 1 × 3

DATE	HOUR	RENTED_BIKE_COUNT
<chr>	<dbl>	<dbl>
19/06/2018	18	3556

All-time high bike rental on 19 June 2018 during the 18th hour of the day.

Hourly popularity and temperature by seasons

A data.frame: 10 × 4

SEASONS	HOUR	AVG_TEMP	AVG_RENTED_BIKE_COUNT
<chr>	<dbl>	<dbl>	<dbl>
Summer	18	29.38791	2135.141
Autumn	18	16.03185	1983.333
Summer	19	28.27378	1889.250
Summer	20	27.06630	1801.924
Summer	21	26.27826	1754.065
Spring	18	15.97222	1689.311
Summer	22	25.69891	1567.870
Autumn	17	17.27778	1562.877
Summer	17	30.07691	1526.293
Autumn	19	15.06346	1515.568

- Bike rental not prominent in Winter
- Bike rental highest between 17th and 22nd hours
- Temperatures between 15 and 30 °C

Rental Seasonality

A data.frame: 4 × 5

SEASONS	AVG_BIKE_COUNT	MIN_BIKE_COUNT	MAX_BIKE_COUNT	STDDEV_BIKE_COUNT
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Autumn	924.1105	2	3298	617.3885
Spring	746.2542	2	3251	618.5247
Summer	1034.0734	9	3556	690.0884
Winter	225.5412	3	937	150.3374

- Summer clearly show highest demand
- Winter shows least use of bike

Weather Seasonality

A data.frame: 4 × 10

SEASONS	AVG_TEMPERATURE	AVG_HUMIDITY	AVG_WIND_SPEED	AVG_VISIBILITY	AVG_DEW_POINT_TEMP	AVG_SOLAR_RADIATION	AVG_RAINFALL	AVG_SNOWFALL	AVG_BIKE_COUNT
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Summer	26.587711	64.98143	1.609420	1501.745	18.750136	0.7612545	0.25348732	0.00000000	1034.0734
Autumn	13.821580	59.04491	1.492101	1558.174	5.150594	0.5227827	0.11765617	0.06350026	924.1105
Spring	13.021685	58.75833	1.857778	1240.912	4.091389	0.6803009	0.18694444	0.00000000	746.2542
Winter	-2.540463	49.74491	1.922685	1445.987	-12.416667	0.2981806	0.03282407	0.24750000	225.5412

- Average temperature, humidity, dew point temperature are directly proportional to average bike demands
- Rainfall and wind speed higher in spring than autumn despite comparable temperature and humidity

Bike-sharing info in Seoul

A data.frame: 1 × 6

CITY	COUNTRY	LAT	LNG	POPULATION	TOTAL_BICYCLES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Seoul	Korea, South	37.5833	127	21794000	20000

- Total Bike count for the city of Seoul in South Korea is 20000
- Located at longitude 127° latitude 37.58° and about 21.8 million population

Cities similar to Seoul

A data.frame: 9 × 6

CITY	COUNTRY	LAT	LNG	POPULATION	TOTAL_BICYCLES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Beijing	China	39.9050	116.3914	19433000	16000
Milan	Italy	45.4669	9.1900	1351562	17000
Milan	United States	35.9126	-88.7554	7201	17000
Milan	United States	42.0816	-83.6853	7778	17000
Ningbo	China	29.8750	121.5492	7639000	15000
Seoul	Korea, South	37.5833	127.0000	21794000	20000
Shanghai	China	31.1667	121.4667	22120000	19165
Weifang	China	36.7167	119.1000	9373000	20000
Zhuzhou	China	27.8407	113.1469	3855609	20000

- Quite a number of Chinese cities compares to Seoul's in bike sharing
- 3 other cities called Milan, 2 in US and 1 in Italy with similar bike demand count
- US cities need further investigation

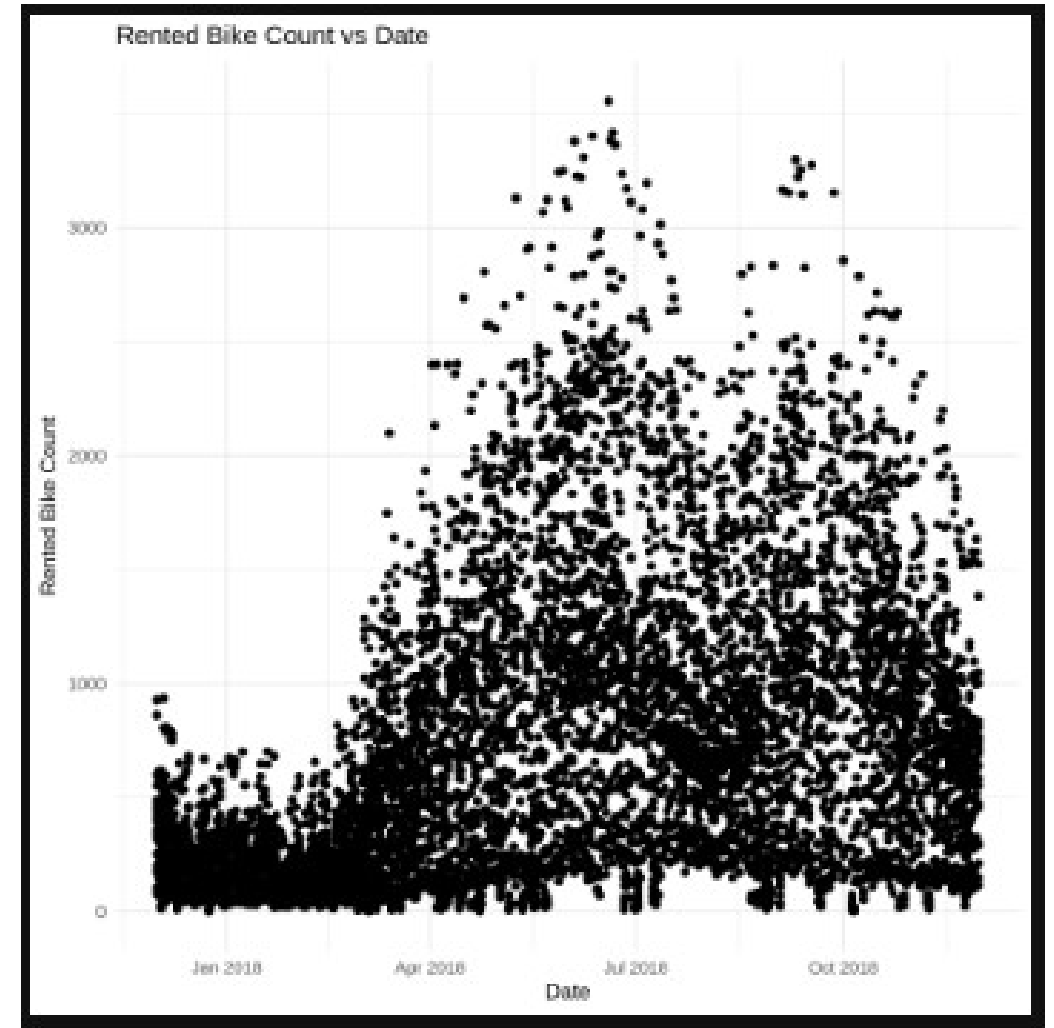
EDA with Visualization

Generated with ggplot2 library. See appendix.

Bike rental vs. Date

Scatter plot of RENTED_BIKE_COUNT vs. DATE

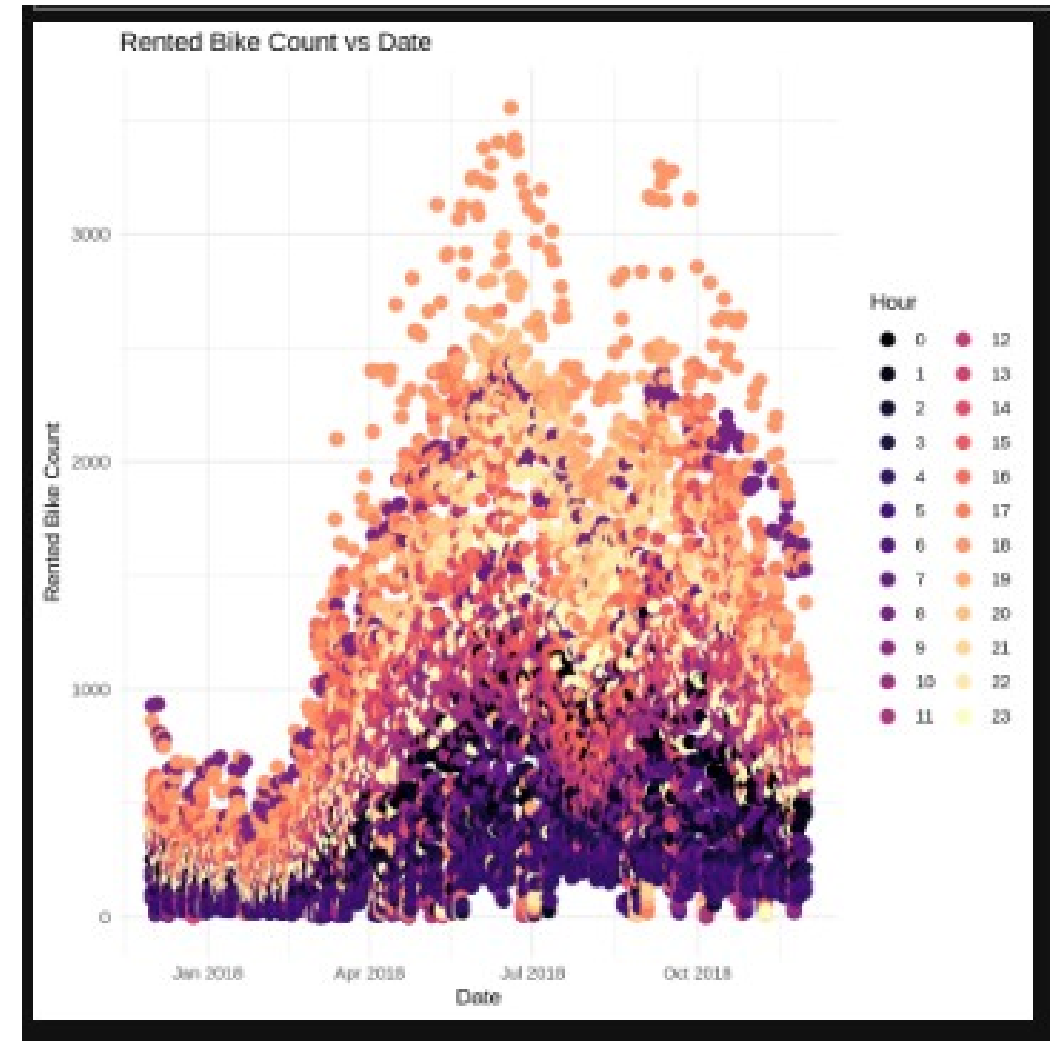
Showing higher demands between May and July, then September and November. 2 Peak periods being June followed by October.



Bike rental vs. Datetime

Scatter plot of RENTED_BIKE_COUNT vs Date
coloured by HOURS

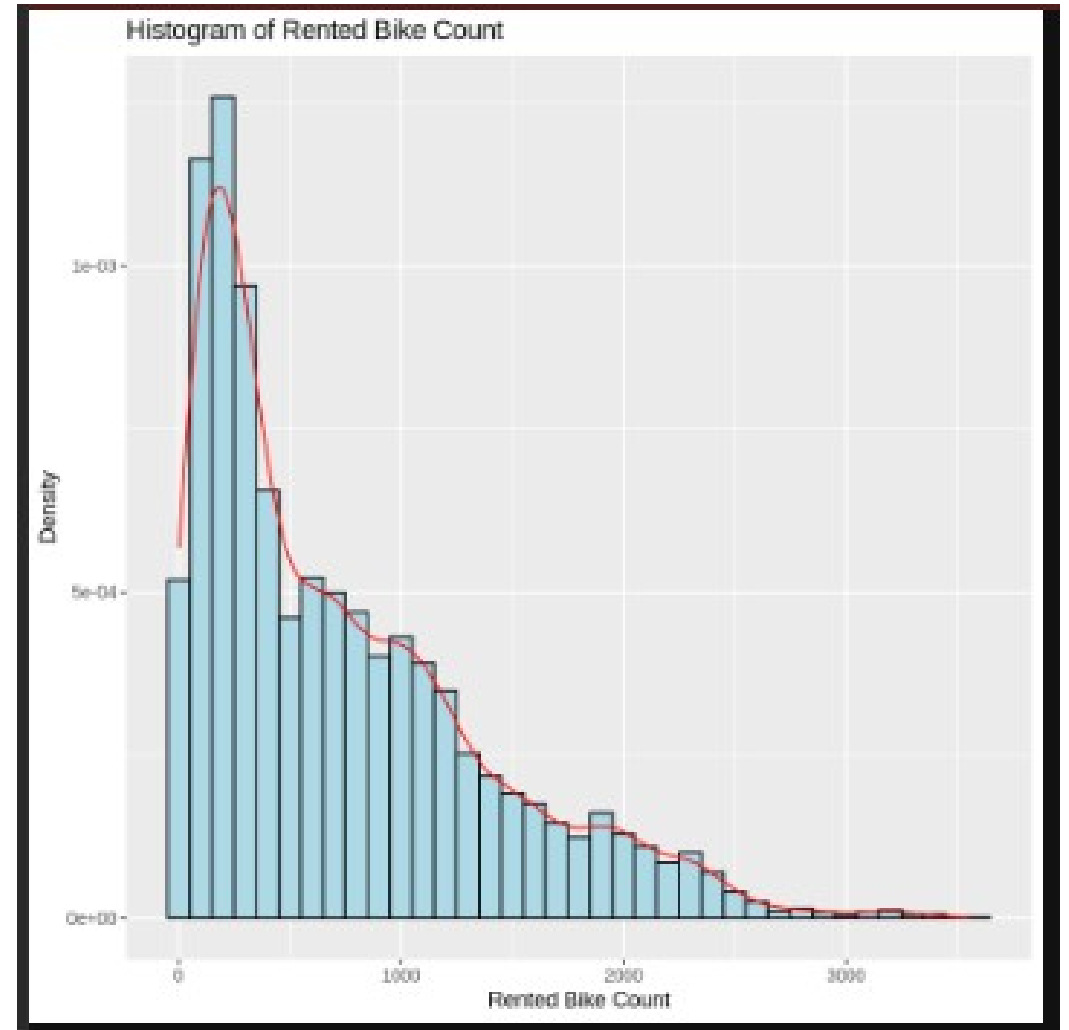
Clearly showing higher patronage of bike-sharing services in hours with brighter colours; afternoon, evening and night.



Bike rental histogram

Histogram of Rented Bike Count overlaid with a kernel density curve of same.

Indicating bike rental counts of between 50 to 150 more frequent and those over 2500 even very rare.

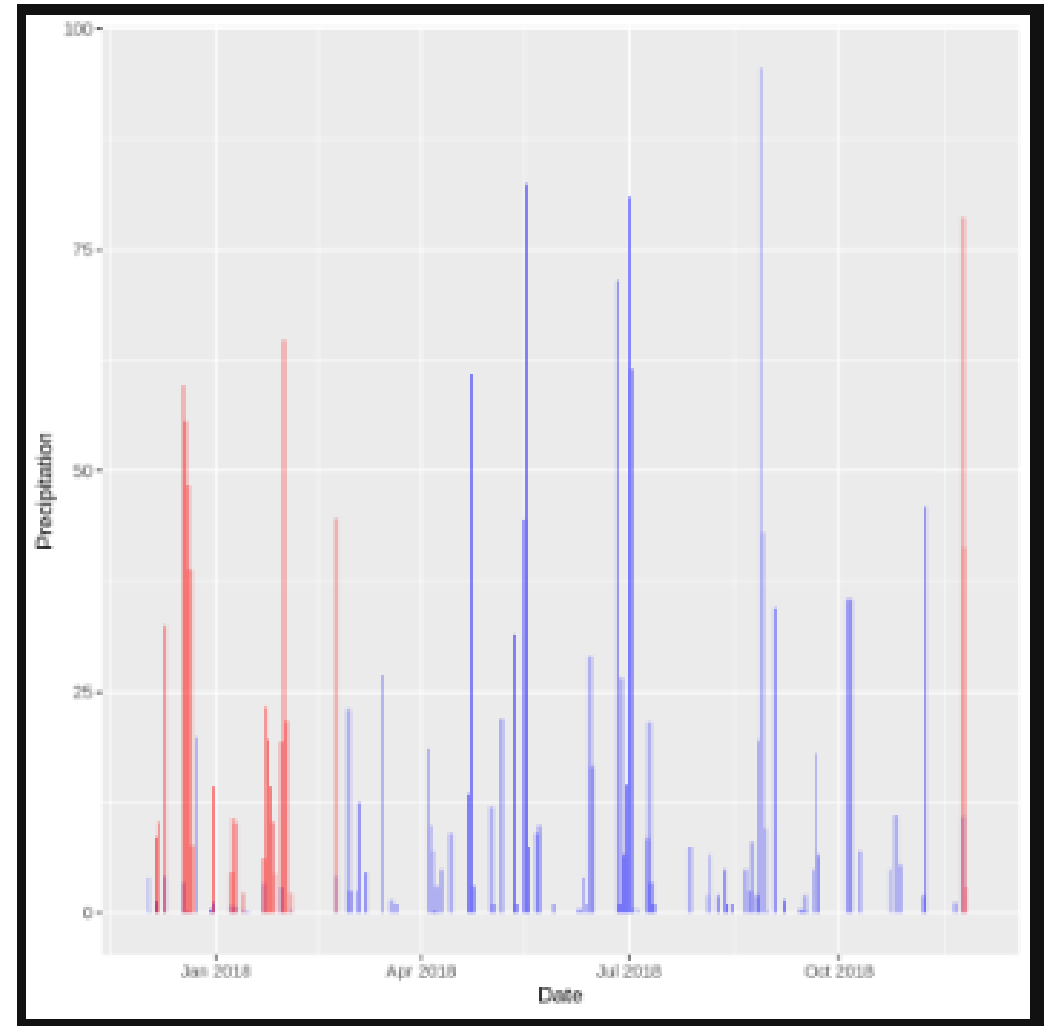


Daily total rainfall and snowfall

Bar chart of daily total rainfall (blue) and snowfall (red)

Snowfall most prominent between December and February.

Rainfall prominent in other months of the year.



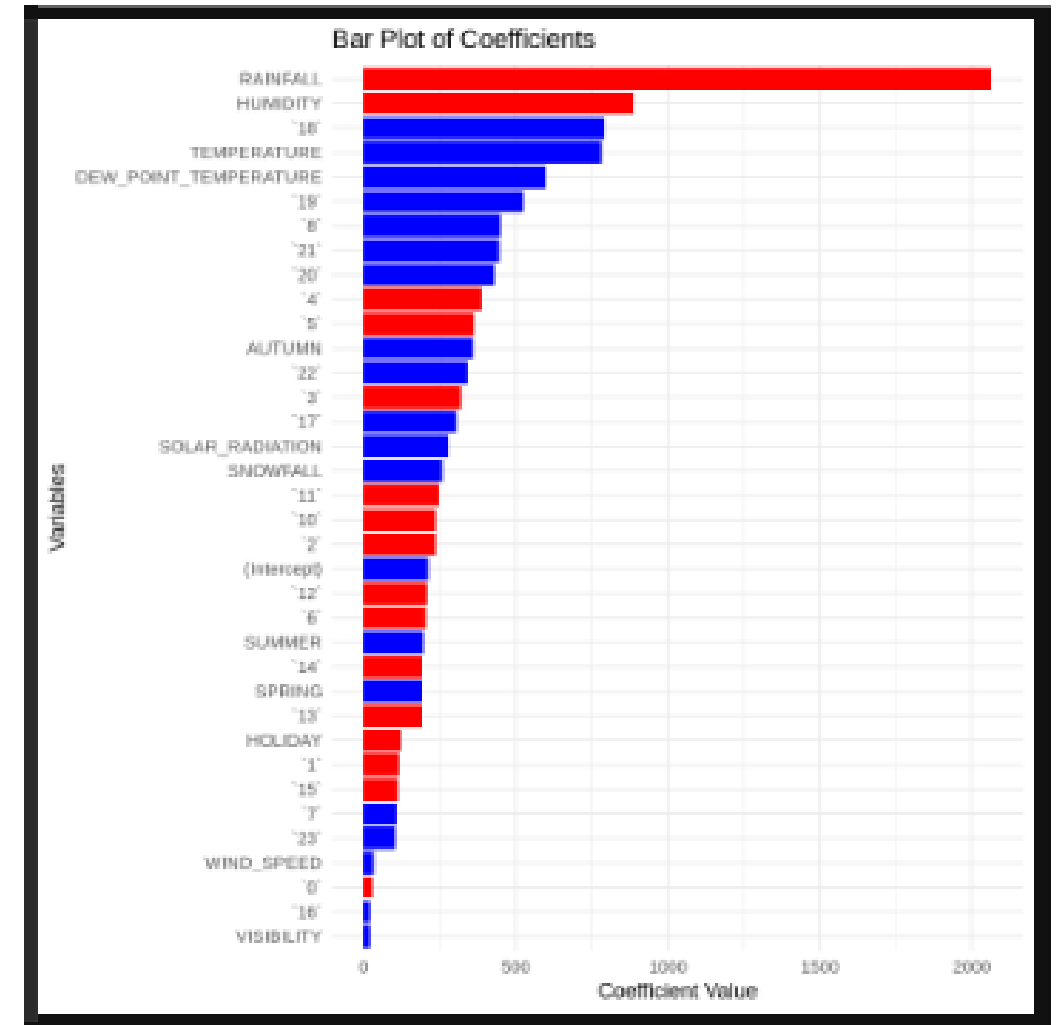
Predictive analysis

Putting Tidymodels and glmnet to work.

Ranked coefficients

Bar chart plot of linear model coefficients ranked according to impact and coloured by direction of impact, red being negative or inverse proportionality and blue positive or direct proportionality

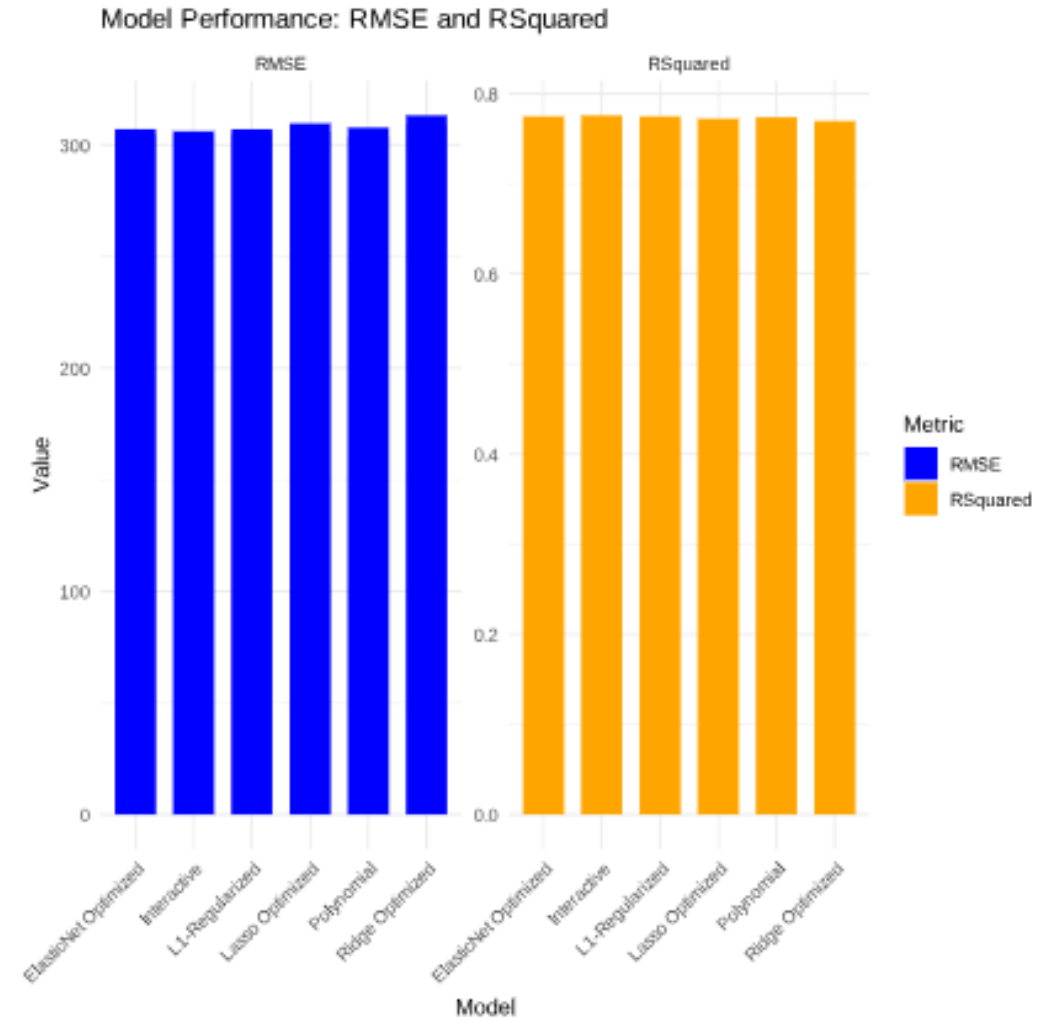
Clearly, rainfall has the most impact on bike rental followed by humidity both affecting negatively. 18th hour being hour with highest bike rental impacts positively, so as temperatures.



Model evaluation

Built and evaluated 6 different models using polynomial terms, interaction terms, regularizations, cross-validation and fine-tuning

All models meet the required performance standards of less than 300 RMSE and higher than 0.72 R-squared but L1-Regularized performed best with both training and test data sets.



Find the best performing model

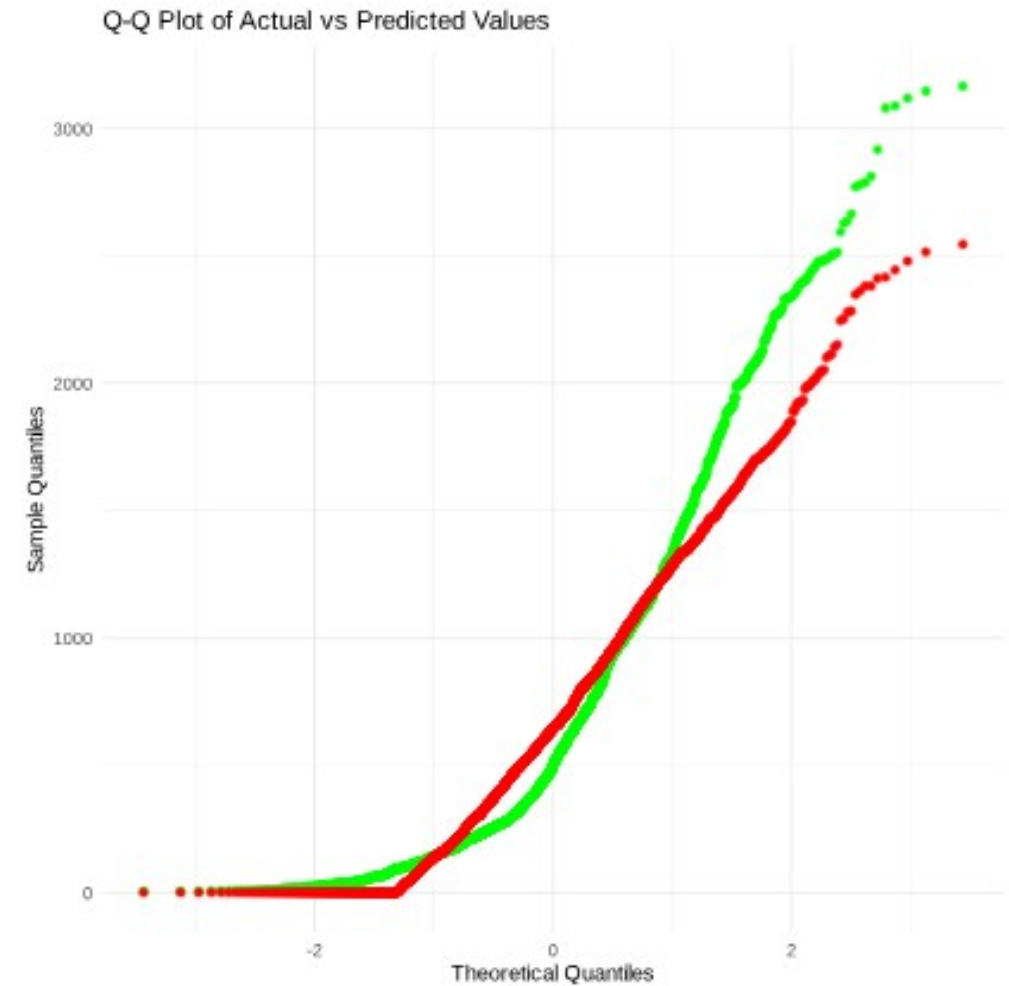
A data.frame: 6 × 5

Model	Train_RMSE	Train_RSquared	Test_RMSE	Test_RSquared
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Polynomial	307.7827	0.7738107	303.1786	0.7725408
Interactive	306.1651	0.7759044	304.8763	0.7695645
L1-Regularized	307.0753	0.7749039	304.2959	0.7707913
Lasso Optimized	309.6409	0.7723009	305.7678	0.7698188
Ridge Optimized	313.2584	0.7696418	308.8523	0.7677715
ElasticNet Optimized	307.1024	0.7748735	304.3183	0.7707524

- Best performing model is L1-Regularized:
 - RMSE 307.08 and 304.3 on train and test data respectively.
 - R-squared 0.775 and 0.771 for train and test data respectively.

Q-Q plot of the best model

Q-Q plot of the best model's test results
vs the truths

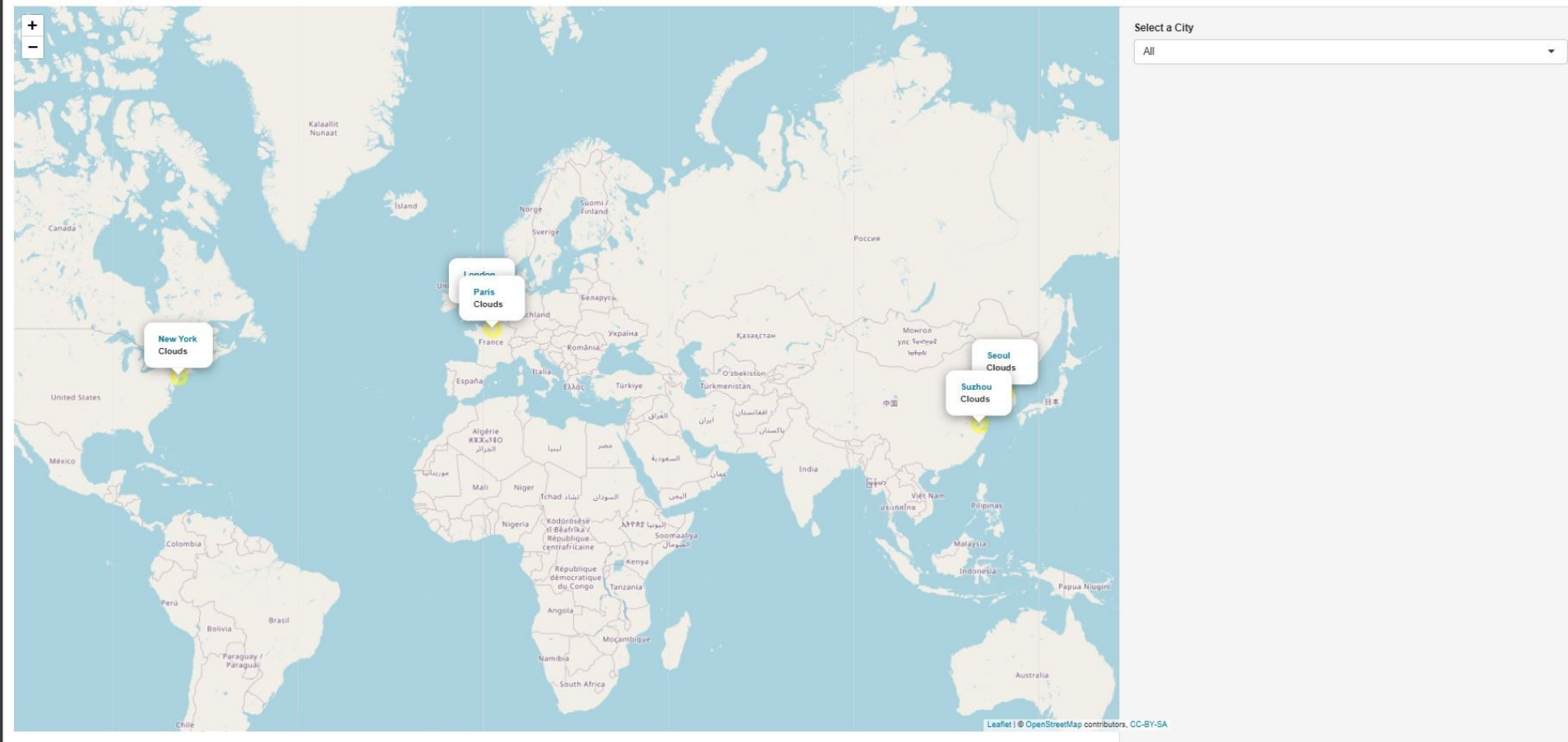


Dashboard

App built using shiny ui and server.

Dashboard Landing Page

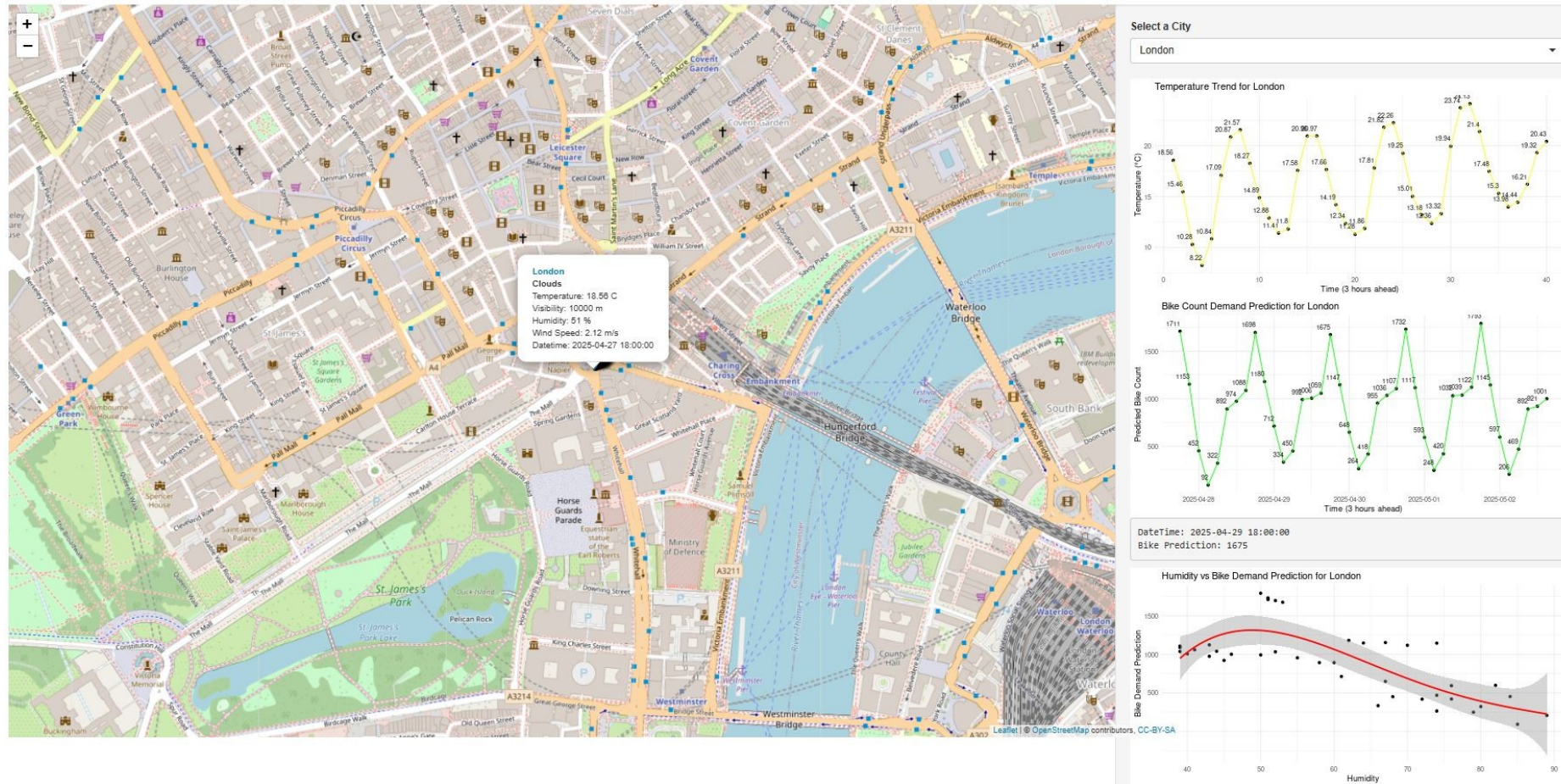
Bike sharing demand forecast from 2025-04-27 to 2025-05-02



Landing page of Shiny App showing popup of choice cities on the world map

London in focus

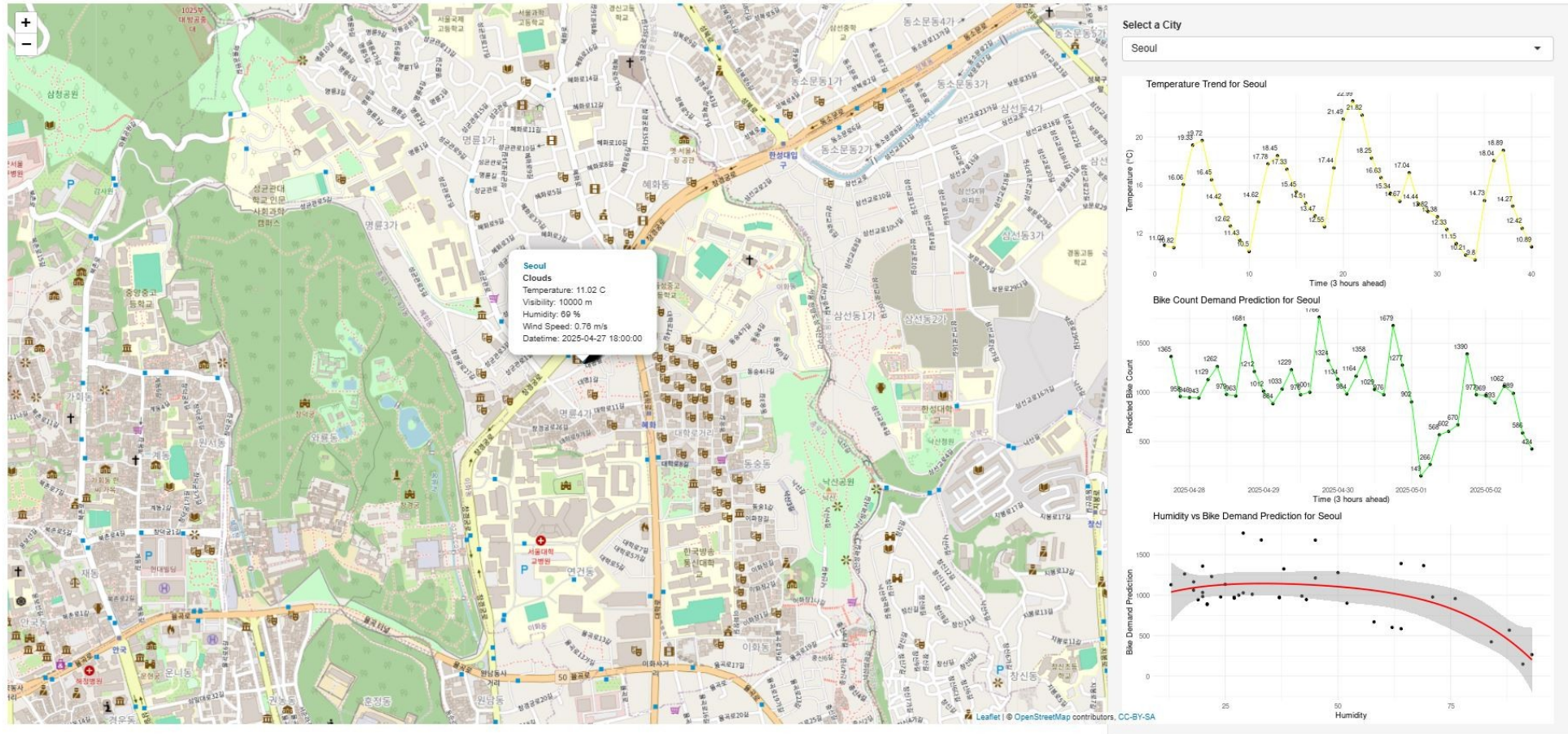
Bike sharing demand forecast from 2025-04-27 to 2025-05-02



Interactive map with visualization of temperature trend, bike demand predictions with textbox showing click point on plot and humidity vs bike demand prediction. Trend line for next 5 days.

Seoul selected

Bike sharing demand forecast from 2025-04-27 to 2025-05-02



Interactive map with popup showing several weather. Trend lines of temperature and bike demand predictions and humidity vs bike demand. No textbox as no click on bike demand plot.

CONCLUSION



- Bike-share heavily influenced by weather especially temperature and humidity
- Bike-share influenced by time of year and day
- More Asian cities depend on bike-share
- Bike-share not only depend on simple weather and time variables but also on polynomials and interaction of the variables
- Regularization, cross-validation and fine-tuning improves model performance.

APPENDIX



- API Call
- Web-scraping
- Data Wrangling
- SQL EDA
- GGPlot2 EDA
- Regression Models
- Shiny App

API Call

```
json_list <- content(response, as = "parsed")
# Note that the 5-day forecast JSON result is a List of Lists. You can print the response to check the results
results <- json_list$list
handle_null <- function(x) {
  if (is.null(x)) 10000 else x
}
# Loop the json result
for(result in results) {
  city <- c(city, city_name)
  weather <- c(weather, result$weather[[1]]$main)
  visibility <- c(visibility, handle_null(result$visibility))
  temp <- c(temp, result$main$temp)
  temp_min <- c(temp_min, result$main$temp_min)
  temp_max <- c(temp_max, result$main$temp_max)
  pressure <- c(pressure, result$main$pressure)
  humidity <- c(humidity, result$main$humidity)
  wind_speed <- c(wind_speed, result$wind$speed)
  wind_deg <- c(wind_deg, result$wind$deg)
  date_time <- as.POSIXct(result$dt, origin = "1970-01-01", tz = "UTC")
  forecast_datetime <- c(forecast_datetime, result$dt_txt) # format(date_time, "%Y-%m-%d %H:%M:%S")
  month <- as.numeric(format(date_time, "%m"))
  if (month %in% c(12, 1, 2)) {
    season <- c(season, "Winter")
  } else if (month %in% c(3, 4, 5)) {
    season <- c(season, "Spring")
  } else if (month %in% c(6, 7, 8)) {
    season <- c(season, "Summer")
  } else if (month %in% c(9, 10, 11)) {
    season <- c(season, "Fall")
  }
}
```

Code snippet to populate several lists of weather parameters in API call.

Webscraping

```
[2]: url <- "https://en.wikipedia.org/wiki/List_of_bicycle-sharing_systems"
# Get the root HTML node by calling the `read_html()` method with URL
wiki_tables <-html_nodes(read_html(url), "table")
wiki_tables

{xml_nodeset (4)}
[1] <table class="wikitable sortable sticky-header" style="background:#f8f9fa ...
[2] <table class="nowraplinks mw-collapsible autocollapse navbox-inner" style ...
[3] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbod ...
[4] <table class="nowraplinks navbox-subgroup" style="border-spacing:0"><tbod ...
```

Read HTML table nodes from Wiki page

```
[4]: # Convert the bike-sharing system table into a dataframe
bike_share <- html_table(wiki_tables[[1]])
bike_share
```

Extract bike sharing systems HTML table from tables

Data Wrangling

Convert categorical variables to numerical variables and delete categorical variables.

```
[13]: # Convert SEASONS, HOLIDAY, FUNCTIONING_DAY, and HOUR columns into indicator columns.
bike_sharing_df <- bike_sharing_df %>%
  mutate(
    SPRING = ifelse(SEASONS == "Spring", 1, 0),
    SUMMER = ifelse(SEASONS == "Summer", 1, 0),
    AUTUMN = ifelse(SEASONS == "Autumn", 1, 0),
    WINTER = ifelse(SEASONS == "Winter", 1, 0),
    HOLIDAY = ifelse(HOLIDAY == "Yes", 1, 0),
    FUNCTIONING_DAY = ifelse(FUNCTIONING_DAY == "Yes", 1, 0),
    H00 = ifelse(HOUR == 0, 1, 0),
    H01 = ifelse(HOUR == 1, 1, 0),
    H02 = ifelse(HOUR == 2, 1, 0),
    H03 = ifelse(HOUR == 3, 1, 0),
    H04 = ifelse(HOUR == 4, 1, 0),
    H05 = ifelse(HOUR == 5, 1, 0),
    H06 = ifelse(HOUR == 6, 1, 0),
    H07 = ifelse(HOUR == 7, 1, 0),
    H08 = ifelse(HOUR == 8, 1, 0),
    H09 = ifelse(HOUR == 9, 1, 0),
    H10 = ifelse(HOUR == 10, 1, 0),
    H11 = ifelse(HOUR == 11, 1, 0),
    H12 = ifelse(HOUR == 12, 1, 0),
    H13 = ifelse(HOUR == 13, 1, 0),
    H14 = ifelse(HOUR == 14, 1, 0),
    H15 = ifelse(HOUR == 15, 1, 0),
    H16 = ifelse(HOUR == 16, 1, 0),
    H17 = ifelse(HOUR == 17, 1, 0),
    H18 = ifelse(HOUR == 18, 1, 0),
    H19 = ifelse(HOUR == 19, 1, 0),
    H20 = ifelse(HOUR == 20, 1, 0),
    H21 = ifelse(HOUR == 21, 1, 0),
    H22 = ifelse(HOUR == 22, 1, 0),
    H23 = ifelse(HOUR == 23, 1, 0)
  ) %>%
  select(-SEASONS, -HOLIDAY, -FUNCTIONING_DAY, -HOUR)
```

SQL EDA

```
[42]: SQL_QUERY <- "  
SELECT  
  SEASONS,  
  AVG(TEMPERATURE) AS AVG_TEMPERATURE,  
  AVG(HUMIDITY) AS AVG_HUMIDITY,  
  AVG(WIND_SPEED) AS AVG_WIND_SPEED,  
  AVG(VISIBILITY) AS AVG_VISIBILITY,  
  AVG(DEW_POINT_TEMPERATURE) AS AVG_DEW_POINT_TEMP,  
  AVG(SOLAR_RADIATION) AS AVG_SOLAR_RADIATION,  
  AVG(RAINFALL) AS AVG_RAINFALL,  
  AVG(SNOWFALL) AS AVG_SNOWFALL,  
  AVG(RENTED_BIKE_COUNT) AS AVG_BIKE_COUNT  
FROM  
  SEOUL_BIKE_SHARING  
GROUP BY  
  SEASONS  
ORDER BY  
  AVG_BIKE_COUNT DESC;"
```

SQL to generate weather seasonality

SQL EDA - cont'd

```
[41]: SQL_QUERY <- "SELECT SEASONS, AVG(RENTED_BIKE_COUNT) AVG_BIKE_COUNT, MIN(RENTED_BIKE_COUNT) MIN_BIKE_COUNT,
      MAX(RENTED_BIKE_COUNT) MAX_BIKE_COUNT, SQRT(AVG(RENTED_BIKE_COUNT * RENTED_BIKE_COUNT) - AVG(RENTED_BIKE_COUNT) *
      AVG(RENTED_BIKE_COUNT)) STDDEV_BIKE_COUNT FROM SEOUL_BIKE_SHARING GROUP BY SEASONS;"

dbGetQuery(conn, SQL_QUERY)
```

SQL to generate rental seasonality

```
[40]: SQL_QUERY <- "WITH SEASONALAVGS AS (SELECT SEASONS, HOUR, AVG(TEMPERATURE) AVG_TEMP,
      AVG(RENTED_BIKE_COUNT) AVG_RENTED_BIKE_COUNT FROM SEOUL_BIKE_SHARING GROUP BY SEASONS, HOUR)
      SELECT SEASONS, HOUR, AVG_TEMP, AVG_RENTED_BIKE_COUNT FROM SEASONALAVGS
      ORDER BY AVG_RENTED_BIKE_COUNT DESC LIMIT 10;"

dbGetQuery(conn, SQL_QUERY)
```

SQL to generate Hourly popularity and temperature by season

```
[38]: SQL_QUERY <- "SELECT DATE, HOUR, RENTED_BIKE_COUNT FROM SEOUL_BIKE_SHARING
      WHERE RENTED_BIKE_COUNT = (SELECT MAX(RENTED_BIKE_COUNT) FROM SEOUL_BIKE_SHARING)"

dbGetQuery(conn, SQL_QUERY)
```

SQL to generate all-time high demand

SQL EDA - cont'd

```
[49]: SQL_QUERY <- "  
SELECT  
  W.CITY,  
  W.COUNTRY,  
  W.LAT,  
  W.LNG,  
  W.POPULATION,  
  SUM(B.BICYCLES) AS TOTAL_BICYCLES  
FROM  
  WORLD_CITIES W,  
  BIKE_SHARING_SYSTEMS B  
WHERE  
  W.CITY = 'Seoul' AND  
  W.CITY = B.CITY  
GROUP BY  
  W.CITY,  
  W.COUNTRY,  
  W.LAT,  
  W.LNG,  
  W.POPULATION;  
"  
dbGetQuery(conn, SQL_QUERY)
```

Total Bike Count and City Info
for Seoul

```
[50]: SQL_QUERY <- "  
SELECT  
  W.CITY,  
  W.COUNTRY,  
  W.LAT,  
  W.LNG,  
  W.POPULATION,  
  SUM(B.BICYCLES) AS TOTAL_BICYCLES  
FROM  
  WORLD_CITIES W  
JOIN  
  BIKE_SHARING_SYSTEMS B  
ON  
  W.CITY = B.CITY  
GROUP BY  
  W.CITY,  
  W.COUNTRY,  
  W.LAT,  
  W.LNG,  
  W.POPULATION  
HAVING  
  SUM(B.BICYCLES) BETWEEN 15000 AND 20000;  
"  
dbGetQuery(conn, SQL_QUERY)
```

Cities with comparable bike scale
to Seoul with coordinates

GGPlot2 EDA

```
[15]: ggplot(seoul_bike_sharing, aes(x = DATE, y = RENTED_BIKE_COUNT, color = HOUR)) +  
      geom_point(size = 3, na.rm=TRUE) + # Add points with a specified size  
      labs(title = "Rented Bike Count vs Date",  
           x = "Date", y = "Rented Bike Count",  
           color = "Hour") + # Label for the color legend  
      theme_minimal() + # Use a minimal theme  
      scale_color_viridis_d(option = "magma")
```

Plotting RENTED_BIKE_COUNT as a time series
with HOURS as the colour

```
[16]: ggplot(seoul_bike_sharing, aes(x = RENTED_BIKE_COUNT)) +  
      geom_histogram(aes(y=..density..), color = "black", fill = "lightblue", binwidth = 100) +  
      geom_density(color = "red") +  
      labs(title = "Histogram of Rented Bike Count", x = "Rented Bike Count", y = "Density")
```

Histogram overlaid with a kernel density
curve

Regression Models

```
lm_recipe <- recipe(RENTED_BIKE_COUNT ~ TEMPERATURE + HUMIDITY + WIND_SPEED + VISIBILITY +  
  DEW_POINT_TEMPERATURE + SOLAR_RADIATION + RAINFALL + SNOWFALL +  
  `0` + `1` + `10` + `11` + `12` + `13` + `14` + `15` + `16` +  
  `17` + `18` + `19` + `2` + `20` + `21` + `22` + `23` + `3` +  
  `4` + `5` + `6` + `7` + `8` + AUTUMN + SPRING + SUMMER +  
  HOLIDAY, data = train_data) %>%  
  step_interact(~ HUMIDITY * TEMPERATURE +  
    HUMIDITY * DEW_POINT_TEMPERATURE +  
    HUMIDITY * RAINFALL +  
    TEMPERATURE * RAINFALL +  
    TEMPERATURE * DEW_POINT_TEMPERATURE +  
    DEW_POINT_TEMPERATURE * RAINFALL) %>%  
  step_poly(TEMPERATURE, HUMIDITY, WIND_SPEED, VISIBILITY, DEW_POINT_TEMPERATURE,  
    SOLAR_RADIATION, RAINFALL, SNOWFALL, degree = 4)
```

- Recipe for model

Regression Models - cont'd

```
[24]: # Build at least five different models using polynomial terms, interaction terms, and regularizations.
build_fit_model <- function(model_type, train_data, lm_recipe) {
  if (model_type == "Lasso") {
    tune_spec <- linear_reg(penalty = tune(), mixture = 1) %>% set_engine("glmnet")
    model_grid <- grid_regular(levels = 50, penalty(range = c(-3, 0.3)))
  } else if (model_type == "Ridge") {
    tune_spec <- linear_reg(penalty = tune(), mixture = 0) %>% set_engine("glmnet")
    model_grid <- grid_regular(levels = 50, penalty(range = c(-3, 0.3)))
  } else if (model_type == "ElasticNet") {
    tune_spec <- linear_reg(penalty = tune(), mixture = tune()) %>% set_engine("glmnet")
    model_grid <- grid_regular(levels = 50, penalty(range = c(-3, 0.3)),
                              mixture(range = c(0, 1)))
  } else {
    stop("Invalid model type. Please choose 'Lasso', 'Ridge', or 'ElasticNet'.")
  }
  model_wf <- workflow() %>% add_model(tune_spec) %>% add_recipe(lm_recipe)
  model_cvfolds <- vfold_cv(train_data, v = 5)
  lm_grid <- tune_grid(model_wf,
                      resamples = model_cvfolds, grid = model_grid)
  show_best(lm_grid, metric = "rmse", n = 5)
  show_best(lm_grid, metric = "rsq", n = 5)
  best_model <- lm_grid %>% select_best(metric = "rmse")
  final_model <- model_wf %>% finalize_workflow(best_model)
  fitted_model <- fit(final_model, data = train_data)
  return(fitted_model)
}
```

Build and fit best model grid-searched from fine-tuned, cross-validated models. Optimized models built from this.

Shiny App

UI code snippet

```
5 # Create a RShiny UI
6 shinyUI(
7   fluidPage(padding=5,
8     titlePanel(textOutput("app_title")),
9     # Create a side-bar layout
10    sidebarLayout(
11      # Create a main panel to show cities on a leaflet map
12      mainPanel(
13        # leaflet output with id = 'city_bike_map', height = 1000
14        leafletOutput("city_bike_map", height = 1000, width = 1600)
15      ),
16      # Create a side bar to show detailed plots for a city
17      sidebarPanel(
18        # select drop down list to select city
19        selectInput("city", width = 720,
20          label = "Select a City",
21          choices = c("All", "Seoul", "New York", "Paris", "Suzhou", "London"),
22          selected = "All"),
23        # Seoul, South Korea, New York, USA, Paris, France, Suzhou, China, and London, UK
24        # Placeholder for displaying additional city details (like plots or info)
25        # plotOutput("city_detail_plot")
26        plotOutput("temp_line", height = 300, width = 720),
27        plotOutput("bike_line", height = 300, width = 720, click = "plot_click"),
28        verbatimTextOutput("bike_date_output"),
29        plotOutput("humidity_pred_chart", height = 300, width = 720)
30      )
31    )
32  )
```


Shiny App - cont'd

Server code
snippet

```
30 city_weather_bike_df <- test_weather_data_generation()
31
32 # Calculate the minimum and maximum FORECASTDATETIME
33 min_date <- min(city_weather_bike_df$FORECASTDATETIME)
34 max_date <- max(city_weather_bike_df$FORECASTDATETIME)
35
36 # Update the app title with the minimum and maximum dates
37 output$app_title <- renderText({
38   title <- paste("Bike sharing demand forecast from",
39                 substr(min_date, 1, 10), "to", substr(max_date, 1, 10))
40 })
41
42 # Create another data frame called `cities_max_bike` with each row containing city location info a
43 # prediction for the city
44 cities_max_bike <- city_weather_bike_df %>%
45   group_by(CITY_ASCII) %>%
46   summarize(
47     LNG = first(LNG),
48     LAT = first(LAT),
49     max_bike = max(BIKE_PREDICTION),
50     BIKE_PREDICTION_LEVEL = first(BIKE_PREDICTION_LEVEL),
51     LABEL = first(LABEL)
52   )
53
54 # Observe drop-down event
55 observeEvent(input$city, {
56   selected_city <- input$city
57   if (selected_city == "All") {
58     # Render a leaflet map with circle markers and popup weather LABEL for all cities
59     output$city_bike_map <- renderLeaflet({
60       leaflet() %>%
61         addTiles() %>%
62         addCircleMarkers(
```

REFERENCES

Sathishkumar V E, Jangwoo Park, and Yongyun Cho.

Using data mining techniques for bike sharing demand prediction in metropolitan city. Computer Communications, Vol.153, pp.353-366, March, 2020

Sathishkumar V E and Yongyun Cho.

A rule-based model for Seoul Bike sharing demand prediction using weather data European Journal of Remote Sensing, pp. 1-18, Feb, 2020

Global Bike Sharing Systems Dataset

https://en.wikipedia.org/wiki/List_of_bicycle-sharing_systems

Open Weather API Data

Thank You!